Part I Front Matter & Governance

Unified Software Requirements Specification (SRS) & Project Plan

Group 18

Itinerary Map

Version 0.1 Baseline

2. Personnel & Roles:

Member Name	Email	Primary technical disciplines
Eric Lee	lee906@mcmaster.ca	Optimization Engine Development
Hashim Butt	butth11@mcmaster.ca	Full-Stack Development
Lihan Zhang	zhanl212@mcmaster.ca	Backend Development
Rebecca Yang	yangr127@mcmaster.ca	Optimization engine
Shaoyuan Wu	wu485@mcmaster.ca	Backend Development
Xueyan Feng	Fengx66@mcmaster.ca	Frontend Development
Yuhao Zhang	zhang169@mcmaster.ca	Frontend Development & Optimization

Team Coordinator: Hashim Butt

3. Definitions, Acronyms & Abbreviations:

Itinerary: A structured travel schedule containing planned attractions, restaurants, activities, travel routes, and estimated times for a trip.

FE (Frontend): The client-side portion of the application responsible for the user interface and user interactions.

BE (Backend): The server-side portion of the application responsible for APIs, business logic, data processing, and database access.

POI: Stands for “Point of Interest”: A visitable location considered for itinerary generation, including attractions, restaurants, museums, parks, and shops. Accommodation locations are treated separately as fixed starting and ending points

Optimization Engine: A separate logical component that uses candidate POIs, routing information, and user constraints to generate an ordered and scheduled itinerary.

PoC (Proof of Concept): An early working implementation used to demonstrate technical feasibility and validate major system risks.

WCAG: Web Content Accessibility Guidelines used as a reference for web accessibility requirements.

4. Contributions Table:

Member Name	Sections Authored	Sections Reviewed	Percentage
Eric Lee	6, 7	5	
Hashim Butt	13, 17	2, 3, 8	
Lihan Zhang	5, 11	5, 11	7%
Rebecca Yang	9, 16	8	
Shaoyuan Wu	1, 2, 3, 4, 8, 12	9, 10, 13	20%
Xueyan Feng	10	6, 7	10%
Yuhao Zhang	14, 15	11, 12, 17	13%

5. Executive Summary & Purpose:

When people plan a trip, they usually start by saving places they are interested in a high rated restaurant, a well-known city landmark, or a concept store that has recently gone viral. These end up spreading across social media, map bookmarks and notes, but they do not always come up with a detailed and reasonable route. You must check opening hours, work out how long it takes to get from one place to the next, and keep everything within your budget and the number of days you have. Many people spend hours just trying to build their schedule by hand, trying to merge different routes they have found into one, and the result can include wasted backtracking or stops that are already closed when you arrive.

Itinerary Map is a web application that aims to do this planning for the user. Users configure a trip and view candidate places on an interactive map. From those places, the system generates a day-by-day itinerary. It represents the destinations and the travel connections between them as a graph, and uses optimization algorithms to find a schedule that fits the user's available time, travel distance, opening hours and budget, while trying to keep as many of their preferred places as possible. Users can then edit the itinerary and have it recalculated.

Since an itinerary has no single correct answer, the project also compares different generation approaches against each other. The comparison looks at how efficient the travel is, how well the constraints are satisfied, and how closely the result matches what the user asked for.

The goal is to help a user go from a list of saved places to a plan they can actually follow, with much less time spent on logistics. For the team, the project is mainly about constrained route optimization, together with an interface that aims to make the results easy to use and adjust.

Part II Software Requirements Specification

6. Stakeholders & User Personas

- **Primary Clients**

Travelers & Tourists: The direct consumers of the application who use it to discover POIs, build collections, and execute generated schedules.

Academic Evaluators: People who assess the application and the steps towards forming it.

- Secondary Stakeholders

Destination POIs & Local Businesses: Museums, historic sites, parks, and restaurants whose operational constraints (opening hours, capacity limits, mandatory booking windows, ticket costs) directly affect the graph optimization constraints.

Third-Party Data & Service Providers: An external service provider supplying foundational mapping layers, Place API metadata (e.g., attraction coordinates, operating hours, dwell-time indicators), and routing/distance matrix data. (For this project, it will be Google Maps).

- End-user Archetypes

The Efficient Explorer: Seeks to visit the maximum number of high-priority attractions in limited time; requires strict time-window adherence, minimal transit downtime, and exact sequencing.

The Cost Optimizer: Plans under strict financial caps; prioritizes free/discounted POIs, low-cost dining, and walking or public transit routes over private transport.

The Flexible Traveler: Requires low-density daily schedules, large buffer times between activities, geographical clustering within walkable neighborhoods, and built-in flexibility for spontaneous detours.

The Group Coordinator: Plans for multiple people with varying physical stamina and dietary needs; requires predictable meal windows, mobility-friendly routes, and exportable, shareable itineraries.

- User Personas

Dennis (The Efficient Explorer): Dennis is a 29-year-old operation analyst that likes to travel alone twice every year. He approaches travel with enthusiasm and analytical precision. When visiting a new city or town, his objective is near-total coverage: he wants to experience every notable museum, historic site, architectural landmark, viewpoint, and local food spot possible within his available time. He treats trip planning as an optimization puzzle, gathering recommendations from blogs, social media, and travel forums weeks before departure.

Jack (The Group Coordinator): Jack is a 27-year-old project manager that likes to go on trips with his 3 close friends every year. He has naturally inherited the role of the "group trip organizer" among his circle of four close friends. While he genuinely enjoys planning and finding great spots, orchestrating an itinerary for four distinct personalities, budgets, and energy levels can feel like a full-time job. His primary objective is creating a trip everyone enjoys without succumbing to "decision paralysis" or spending half the vacation debating where to eat

next. He wants a balanced plan that keeps the group moving smoothly while leaving room for casual fun and downtime.

7. Constraints & Assumptions

- Constraints

Client-Side Hardware: Minimum dual-core CPU with 4 GB RAM. WebGL-compatible GPU hardware acceleration is strictly required to render interactive map layers smoothly. Responsive viewport support spanning a minimum resolution of 375 × 667 (mobile) up to 3840 × 2160 (4K desktop).

Operating System & Browsers: Windows 10/11, macOS 12+, Ubuntu 20.04+ (or equivalent Linux distributions). iOS 15+ (Safari / WebKit engine) and Android 10+ (Chrome / Blink engine). Evergreen desktop and mobile browsers updated within the last two major versions: Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.

Google Maps API Cost: System operation must not go above the monthly budget cap of the $200 monthly credit provided by Google.

Cloud Infrastructure Cost: Deployment relies on student/academic tier cloud infrastructure (e.g., AWS Free Tier, GCP Cloud Run, Vercel, or Render).

Web-Based Distributed Architecture: The system must operate as a decoupled web application (frontend client communicating with a backend API service over TLS 1.3 / HTTPS). A native mobile client is not required for the core web MVP and is treated as a P3 extension.

No Permanent Storage: Place Details (e.g., names, reviews, phone numbers, operating hours) cannot be stored permanently in the application database. Only Google Place IDs, latitude, and longitude coordinates may be cached, subject to Google's 30-day refresh policy.

No Alternative Layering: Google Maps POI data cannot be rendered or overlaid onto non-Google base maps (such as Mapbox or OpenStreetMap tiles).

PIPEDA: As the application operates in Ontario under Canadian federal jurisdiction, it must adhere to the 10 fair information principles under PIPEDA. Explicit opt-in consent must be gathered for collecting personal identity data (email, name) and usage data. Data collection must be limited strictly to what is necessary for generating itineraries. Users must have the self-service capability to inspect, correct, and delete their stored personal travel data.

GDPR: The application must follow the GDPR to allow European travelers to use the service internationally. User account deletion trigger must execute a complete wipeout of all associated data like user profile data, saved collections, and generated schedules across relational tables.

McMaster/MILO intellectual property considerations: Under McMaster University's IP Policy administered by the McMaster Industry Liaison Office (MILO), undergraduate students generally retain sole ownership of the Foreground IP developed during capstone coursework.

- Assumptions

Continuous Network Connectivity: The system assumes constant, stable client-side internet access (minimum 3G mobile broadband or 5 Mbps Wi-Fi) during schedule generation.

Third-Party API Availability & Accuracy: The system assumes that Google Maps API maintains an operational service availability and Google Places metadata provides reasonably accurate, up-to-date operating hours and geographic coordinates for queried attractions.

Transit Mode Uniformity: Travel duration estimations assume standard transit conditions (standard driving, walking or public transit speeds provided by the Google Routes API) without real-time micro-disruptions such as subway rail delays or temporary sidewalk closures.

Language & Locale: The platform UI and geocoding input handling are initially scoped to English-language localization and standard Gregorian calendar formatting.

8.Functional Requirements

P0 Must-Have Requirements:

Trip Configuration Input: The user's destination city, travel dates, budget, daily start and end times, travel interests, accommodation location, travel pace, and preferred transportation mode. **Expected Output**: A validated trip configuration containing the user's selected travel parameters and preferences. **Data Flow**: User → Trip Configuration Interface → Input Validation → Backend → Trip Configuration Data **Primary Component**: Web Frontend + Backend

POI Retrieval & Filtering

Input: The validated trip configuration from Trip Configuration and geographic search parameters. **Expected Output:** A filtered list of candidate POIs, including attractions and restaurants, that can be used for itinerary generation. **Data Flow**: Trip Configuration Data → Backend → External POI Service → POI and Restaurant Filtering → Candidate POI and Restaurant Data. **Primary Component**: Backend

Automatic Itinerary Generation

Input: The user's validated trip configuration, candidate POIs, accommodation location, estimated visit durations, and travel-time information. **Expected Output**: A feasible day-by-day itinerary containing an ordered sequence of attractions, restaurants, and travel periods with scheduled start and end times. **Data Flow**: Trip Configuration Data + Candidate POI + Travel-Time Data → Itinerary Generation Engine → Constraint

Validation → Generated Itinerary. **Primary Component**: Backend + Optimization Engine

Route & Travel-Time Planning

Input: Candidate POI locations, accommodation location, and selected transportation mode. **Expected Output**: Route and travel-time information between relevant locations for use by the itinerary generation engine. **Data Flow**: Itinerary Locations → Backend → External Routing Service → Route and Travel-Time Data → Itinerary Generation Engine. **Primary Component**: Backend

Itinerary & Interactive Map Visualization

Input: The generated itinerary, POI coordinates, scheduled activity times, estimated travel times, and route information. **Expected Output**: A web-based day-by-day itinerary displays scheduled activities and travel periods, together with an interactive map showing itinerary locations, their sequence, and corresponding routes. **Data Flow**: Generated Itinerary + POI and Route Data → Backend API → Web client → Itinerary View + Interactive Map. **Primary Component**: Web Frontend

P1 High Priority:

Trip Saving & Loading

Input: The user's trip configuration and generated or modified itinerary. **Expected Output**: A persistently stored trip that can be retrieved and reopened by the user in a future application session. **Data Flow**: Trip Configuration + Itinerary → Web Client → Backend → Persistent Storage → Saved Trip Data → Web Client. **Primary Component**: Web Frontend + Backend

Interactive Itinerary Editing & Recalculation

Input: The existing itinerary and user modifications, including deleting an activity, changing an activity's duration, reordering activities, or locking selected activities. **Expected Output**: An updated and feasible itinerary reflecting the user's modifications, with affected schedules and travel times recalculated while preserving locked activities whenever possible. **Data Flow**: User Modification → Web Client → Backend → Itinerary and Route Recalculation → Updated Itinerary → Web client. **Primary Component**: Web Frontend + Backend + Optimization Engine

User Authentication & Account Management

Input: The user's registration credentials, login credentials, or account management request. **Expected Output**: An authenticated user session for valid credentials, appropriate error feedback for invalid credentials, and the ability for users to manage or

delete their account information. **Data Flow**: User → Web Client → Backend Authentication Service → User Database → Authentication Result → Web Client. **Primary Component**: Web Frontend + Backend

P2 Medium Priority:

Itinerary Sharing & Export

Input: The user's selected itinerary and selected sharing or export option. **Expected Output:** A shareable or exported representation of the selected itinerary that can be accessed outside the Itinerary Map application. **Data Flow**: Selected Itinerary → Web client → Sharing/Export Function → External Application or Recipient. **Primary Component**: Frontend + Backend

P3 Low Priority:

Mobile Application Support

Input: The user's existing trip data, itinerary information, and account or session data from the web-based system. **Expected Output**: A mobile application that provides access to the core itinerary planning, viewing, and editing features available in the web client. **Data Flow**: User → Mobile Application → Backend API → Itinerary Services → Mobile Application. **Primary Componen**t: Mobile Frontend + Backend

9. Data Strategy & Quantitative Metrics

Data Source:

Itinerary Map will use a combination of 1) user-provided data; 2) data retrieved from Google Maps Platform services; and 3) system-generated test data.

The Google Places API will provide information about attractions and restaurants, such as their names, locations, categories, opening hours, and unique Place IDs. The Google Routes API will provide the estimated time and distance needed to travel between the accommodation and each POI, as well as between the POIs themselves. In the optimization engine, each location will be represented as a node, and each possible route between two locations will be represented as an edge containing its travel time and distance. The optimization engine will combine this information with the user’s schedule, budget, transportation mode, and preferences to generate an itinerary.

PostgreSQL will store information created by the application or provided by users, including user accounts, trip settings, preferences, selected Google Place IDs, activity durations, and saved itineraries. A Google Place ID is a unique identifier for a location and can be saved so that the

system can request updated place information from Google when it is needed. Other information received from Google Maps will only be stored or cached when permitted by Google's terms. The application will also display all required Google Maps and third-party source information.

Data Validation and Synthetic Test Data:

Before optimization, the backend will check that required information is present and valid. This includes time ranges, budgets, transportation modes, coordinates, activity durations, opening hours, and travel times. If important information is missing or uncertain, the system may ask for user confirmation, use a clearly stated default value, or exclude the affected POI. Valid information will be converted into a consistent format for the optimization engine.

The team will create synthetic datasets in JSON or CSV format because live Google Maps information can change. These datasets will contain artificial POIs with fixed locations, opening hours, activity durations, costs, priority values, and travel times. They may be created manually or by a script using fixed random seeds. The test datasets will cover normal itineraries, restrictive opening hours, insufficient time or budget, missing data, widely separated locations, impossible itineraries, and boundary cases. The datasets will be stored in the project repository so every algorithm can be tested using the same inputs.

Baseline and Quantitative Metrics

Metric	Measurement	Success Threshold
Constraint validity	Itineraries satisfy all required rules. (Valid itineraries / all generated itineraries)	100%
Feasible solution rate	Known feasible cases for which an itinerary is found (Feasible cases solved / all known feasible cases)	90%
Infeasible case detection	Impossible cases correctly reported without an invalid itinerary	100%
Generation time	Optimization time excluding external API retrieval	Maximum 10 seconds for 20 POIs; 5-second target
Baseline comparison	Cases where the improved algorithm matches or exceeds the baseline preference score	80%
Travel efficiency	Visit time compared with total visit and travel time (Visit time / (visit time + travel time))	Match or exceed the baseline

All evaluation results will record the algorithm version, dataset version, configuration, and execution environment. This will allow the team to repeat the tests and compare algorithm versions fairly.

10. Non-Functional Requirements

The following non-functional requirements define the expected quality attributes of the Itinerary Map system, covering usability, performance, security, reliability, and maintainability. Measurable targets are included where appropriate to support verification during development and testing.

10.1 Usability & Human-Computer Interaction

ID	Requirement	Target / Standard	Verification
NFR-U1	Responsive Design	The web interface shall remain usable on desktop and mobile browsers, including viewports from 375 × 667 pixels.	Responsive UI testing
NFR-U2	Accessibility	The interface should follow WCAG 2.1 Level AA where practical, including keyboard navigation, readable text, and sufficient contrast.	Accessibility testing
NFR-U3	Input Validation & Feedback	Invalid inputs shall display clear validation messages, and long-running operations shall provide visible loading or status feedback.	UI testing

10.2 Performance, Scalability & Speed

ID	Requirement	Target / Standard	Verification
NFR-P1	Response Time	Standard backend requests should be completed within 2 seconds under normal conditions, excluding external API delays.	API performance testing
NFR-P2	Itinerary Generation	The system shall generate an optimized itinerary containing up to 20 POIs within 10 seconds, with a target completion time of 5 seconds, excluding external API retrieval time.	Optimization benchmark
NFR-P3	Map Performance	Map zooming, panning, and route visualization should remain responsive, targeting 60 FPS on supported hardware where feasible.	UI performance testing

10.3 Security, Authentication & Data Privacy

ID	Requirement	Target / Standard	Verification
NFR-S1	Authorization	Users shall not access or modify another user's private trip information without authorization.	Authorization testing
NFR-S2	Secure Communication	Communication between the deployed frontend and backend shall use HTTPS with TLS 1.3, consistent with Section 7.	Security testing

NFR-S3	API Key Protection	Google Maps API keys, database credentials, and other secrets shall be protected and not committed to public repositories.	Configuration review
NFR-S4	Data Privacy	Users shall be able to view, correct, and delete their stored personal travel data, with consent handled as specified in Section 7.	Privacy and deletion testing

10.4 Reliability, Fault Tolerance & Maintainability

ID	Requirement	Target / Standard	Verification
NFR-R1	External API Reliability	Google Places or Routes API failures shall not cause the entire application to crash.	Failure simulation
NFR-R2	Data Integrity	Failed itinerary generation or recalculation shall not corrupt or overwrite previously saved valid trip data.	Integration testing
NFR-R3	Error Handling	Backend and external API failures shall produce meaningful error messages and allow retrying where appropriate.	Failure testing
NFR-R4	Automated Testing	Core backend services, constraint validation, scheduling, and optimization shall be covered by automated tests.	Automated test results

11. Risk Management & Feasibility Analysis

Risk	Likelihood	Impact	Mitigation
Google Maps API usage exceeds the $200 monthly credit	Medium	High	Cache only Place IDs and coordinates (as permitted), batch distance-matrix requests, and set a usage alert and hard quota in the Google Cloud console
Opening hours or travel times from Google are inaccurate or out of date, producing infeasible schedules	Medium	Medium	Show the data source and last-refresh time to the user, allow manual correction of an activity's time, and re-validate constraints after each edit
The optimization problem becomes too slow as the number of POIs and days grows	Medium	High	Cap candidate POIs per day, start with a simple heuristic baseline, and set a time limit on each solve that returns the best feasible result found so far
Project scope is too large for seven students at about 10 hours per week	Medium	High	Commit to P0 and P1, treat P2 and P3 as stretch goals, build P0 first, and drop stretch goals first if sprint reviews show the team falling behind.

Hosting provider is not yet chosen, which could delay integration testing and the demo	Low	Medium	Decide on a provider before the proof-of-concept demo, and keep the backend containerized so it can move between providers
Privacy requirements (PIPEDA, GDPR) are met late or incompletely, especially full deletion of user data	Low	High	Design the deletion path into the database schema from the start and cover it with an integration test
Merge conflicts or integration problems between frontend, backend and optimization module	Medium	Medium	Agree on the API contract early, enforce PR review and CI checks as described in Section 13

Part III Project Development & Execution Plan

12. Team Communication & Meeting Cadence:

The team will hold a short internal meeting every Friday at 12:00 PM. Each meeting is expected to last approximately 15–30 minutes. These meetings will be used to review individual progress, identify blockers, coordinate upcoming tasks, and confirm short-term priorities.

Outside of the scheduled weekly meetings, the 7 team members will communicate through a dedicated Discord group chat. Team members may use the group to discuss technical issues, implementation questions, scheduling concerns, and other project-related matters as they arise.

Important project decisions made through Discord discussions will be documented in the relevant GitHub issue or included in the next set of meeting notes to ensure that key decisions remain traceable.

Meeting notes will be documented and stored in the /meetings directory of the project GitHub repository. The notes will include the meeting date, attendees, major discussion points, decisions made, assigned tasks, and follow-up items.

The team will also meet with the project supervisor at least twice per academic term and with the assigned TA at least once per term, in accordance with the course requirements.

13. Engineering Workflow and Version Control

The projects repository structure will be as follows:

Path	Description

/src/frontend	User facing client
/src/backend	API server, authentication, unit & integration tests
/src/optimization	Optimization engine for itinerary generation
/docs	Class Deliverables
/meetings	Notes for every team meeting

We will adopt the Git Flow branching strategy for our project while using GitHub Issues for issue tracking. We will have a **main** branch for production ready code, **develop** will be for testing completed features where majority of commits will occur. Branches will have 3 types; feat for feature implementations, fix for bugfixes and docs for documentation. Branch names will be in the form of <type>/<contributor>/<issue#> an example of this is feat/hashim/RDX-4210. The develop branch will be merged into main after each functional requirement is completed and tests as well as before key deadlines. Pull requests will require review from at least one other team member; branch author will address comments and update PR if necessary, and branch can only be merged after at least 1 approval. We will leverage GitHub actions for CI/CD integration, checks including build, lint, integration and unit tests will run for all modules, will block merging if any step fails, checks will rerun after each commit once PR is open.

The weekly internal meeting described in Section 12 will serve as the team's Scrum meeting. The scrum master will lead the meeting, letting everyone discuss their progress on their assigned tasks; the scrum master will rotate every week. We will use a GitHub Projects board to manage the backlog, with columns: ToDo, In Progress, In Review, DEV, PROD to categorize issues.

14. Infrastructure & Compute Resources

The initial version of **Itinerary Map** will be developed as a web-based full-stack application. Development will primarily take place on team members' local machines, while a shared deployment environment will be used for integration testing and the final working system. The specific hosting provider has not yet been finalized and will be selected based on cost, ease of deployment, and compatibility with the chosen frontend, backend, and database technologies.

The system will rely on external geographic services for real-world location and routing information. **Google Places API** is planned for retrieving points of interest such as attractions and restaurants, while **Google Routes API** will provide distance, route, and estimated travel-time information. Because these services are externally hosted, API quotas, availability, and potential usage costs will be considered infrastructure constraints.

A persistent database will be used to store trip information, selected points of interest, user constraints and preferences, generated itineraries, and related application state. **PostgreSQL** is currently planned as the primary database because the system contains structured relationships among trips, POIs, itinerary entries, and user-defined constraints.

The core itinerary generation and optimization components are expected to run on standard CPU resources and do not currently require dedicated GPU computing. The system will therefore be designed to operate on conventional development machines and a standard application server. Additional computing resources may be considered later if advanced features requiring machine learning or other computationally intensive techniques are introduced.

15. Technology Stack & Architectural Rationale

The system will follow a **modular client-server architecture** that separates the user interface, backend services, itinerary optimization logic, persistent data storage, and external API integrations. This separation is intended to simplify development and testing while allowing individual components to be modified or extended independently.

The frontend is planned to use **React with TypeScript**. React is well suited to the highly interactive nature of the application, including trip creation, constraint input, itinerary editing, and map-based visualization. TypeScript provides additional type safety and should help reduce integration errors as the frontend grows in complexity.

The backend is planned to use **Python with FastAPI**. Python is appropriate for the project because the itinerary planning component will require scheduling, constraint handling, and optimization algorithms in addition to standard backend functionality. FastAPI provides a lightweight framework for exposing REST APIs and allows the backend and optimization components to remain within the same Python-based development environment.

The itinerary optimization engine will be implemented as a **separate logical module** rather than being tightly coupled to the frontend or API layer. The module will receive candidate POIs, routing information, and user constraints, and will return an ordered and scheduled itinerary. This modular structure will make it easier to improve the optimization approach, compare alternative algorithms, or add more advanced constraints in later stages of the project.

PostgreSQL is planned for persistent data storage because the application contains strongly related entities such as trips, POIs, constraints, itinerary days, and itinerary entries. **Google Places API** and **Google Routes API** will provide POI information, geographic data, routing, and travel-time estimates, allowing the team to focus on itinerary generation rather than implementing large-scale mapping and routing infrastructure from scratch.

For the first stage of development, the team will focus on a **web-based MVP**. Keeping the frontend separate from the backend will also support future expansion to other platforms. A future mobile application could reuse the same FastAPI backend, database, external API integrations, and optimization engine while replacing the web frontend with a mobile client such as React Native. Similarly, a desktop application could reuse the same core system through a dedicated desktop client or a web-based desktop wrapper.

Unit and integration testing will be incorporated into the development workflow. Unit tests will focus on backend services, constraint validation, scheduling, and optimization logic, while integration tests will verify interactions between the frontend, backend, database, and external APIs. The specific testing frameworks will be finalized together with the implementation stack.

16. Proof of Concept (PoC) Demonstration Plan

The proof of concept will be presented through a two-minute video submitted on November 22 and shown during the mandatory class on November 23.

The first minute will use one slide and voice over to explain the project's problem, purpose, key features, and stakeholders. The second minute will show a screen recording of working code rather than slides.

The code demonstration will show progress on the main P0 workflow. A user will enter basic trip information and select approximately 5–10 POIs in one city. The frontend will send this information to the FastAPI backend, and a basic optimization algorithm will generate a one-day itinerary. The result will be displayed as an ordered schedule and, if ready, on an interactive map. The demonstration may also show how the system responds when no feasible itinerary can be created.

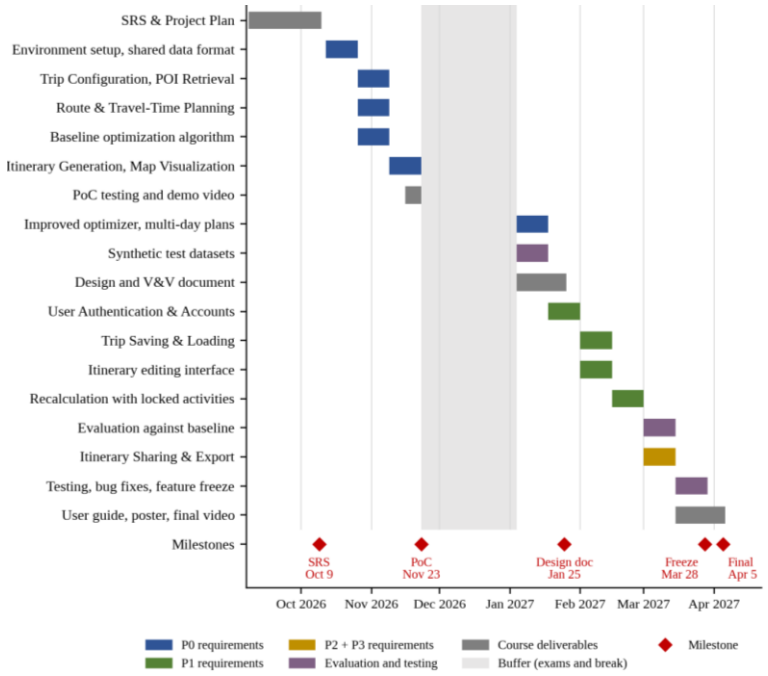
This PoC will test the project's main technical risks: communication between the frontend and backend, Google Maps API access, transfer of data to the optimization engine, basic constraint checking, and itinerary generation. User accounts, mobile support, sharing, and other P1 or P2 features are outside the PoC scope.

Time Period	Planned work
Oct 12 - 25	Set up the frontend, backend, optimization module, API access, and shared data format
Oct 26 - Nov 8	Build the trip form, basic map, backend endpoints, and baseline optimization algorithm
Nov 9 -15	Connect the components and complete the main P0 workflow
Nov 16 -19	Test the PoC, fix major errors, and confirm the final demo
Nov 20, 21	Create the overview slide and record and edit the video
Nov 22	Upload the final two-minute video to Avenue
Nov 23	Present the PoC video

17. Project Schedule and GANTT Chart

Work will be broken down into 3 sub teams; Frontend (Hashim, Xueyan, Yuhao), Backend (Lihan, Shaoyuan) and Optimization Engine (Rebecca, Eric). Since the optimization algorithms are the most complex, the rest of the FE and BE team can help if they have extra capacity. The

fall term delivers the P0 requirements and PoC, followed by a buffer for exams and winter break. The winter term follows with P1 requirements, Design and V&V document. By mid-March-April it includes stretch goals, final fixes and expo prep by April. The optimization engine is scheduled first along with basic FE and BE, due to high priority for PoC due to its complexity, allowing all team members to help with the implementation.



Part IV Appendix

Generative AI was used to summarize and consolidate terminology already present throughout the SRS for #3, Definitions, Acronyms & Abbreviations.

Generated AI was used in #6. Stakeholders & User Personas and 7. Constraints & Assumptions for brainstorming and verifying purposes.

Generated AI was used to brainstorm and explain possible quantitative evaluation metrics for #9. Data Strategy & Quantitative Metrics.

Generated AI was used to assist in creating GANTT chart python script for #17. Project Schedule and GANTT Chart.